ZIP-0803: Encrypted SQLite Replication for Zoo Services

Antje Worring, Zach Kelling*
Zoo Labs Foundation
research@zoo.ngo

January 2026

Abstract

We specify a sidecar-based replication protocol for Zoo Network services that use Hanzo Base (per-org encrypted SQLite). The system streams Write-Ahead Log (WAL) frames through `hanzoai/replicate` to S3-compatible object storage, encrypts at-rest with `hanzoai/age` (X25519 + ChaCha20-Poly1305), and achieves $\leq$ 1 s Recovery Point Objective (RPO) with seconds-scale Recovery Time Objective (RTO). Each pod replicates to a distinct S3 prefix keyed by pod identity, enabling StatefulSet-aware restore. The design reuses the same `ghcr.io/hanzoai/replicate:latest` image deployed across Lux and Zoo ecosystems, requiring zero Zoo-specific code.

1 Motivation

Zoo Network services are migrating from PostgreSQL to Hanzo Base (per-org encrypted SQLite) following the Web5 local-first architecture described in the Zoo research program [1]. SQLite provides sub-microsecond read latency, per-tenant data isolation via deterministic key derivation, and offline-first operation.

However, SQLite's single-writer model means each service instance owns its database file. Without replication:

1. A pod restart loses all data accumulated since the last PersistentVolumeClaim (PVC) snapshot.
2. Disaster recovery requires manual PVC restoration from cloud provider snapshots (minutes to hours).
3. Cross-region failover is impossible without an external replication mechanism.

This proposal introduces continuous WAL streaming to S3 as the replication primitive, providing:

- $\leq$ 1 s RPO via 1-second sync intervals
- Seconds-scale RTO via init-container restore
- At-rest encryption via `hanzoai/age`
- Zero application code changes (sidecar pattern)

2 Architecture

2.1 Components

`hanzoai/replicate` Litestream-based WAL streaming daemon. Watches SQLite WAL, ships frames to S3. Runs as a sidecar container sharing the data volume with the primary service container.

`hanzoai/age` X25519 + ChaCha20-Poly1305 file encryption. Used for encrypting WAL

*zach@zoo.ngo

snapshots and segments at rest in S3. Compatible with `luxfi/age` (same wire format).

`hanzoai/s3` S3-compatible object storage. Deployed as a ClusterIP service within the Zoo K8s cluster (`s3.zoo.svc.cluster.local:9000`). Backed by DigitalOcean Spaces or MinIO depending on environment.

ConfigMap Shared `replicate-config` ConfigMap containing the Litestream YAML configuration. All services reference the same ConfigMap; pod identity (`POD_NAME`) provides per-replica S3 path namespacing.

2.2 Data Flow

```
Service writes to SQLite
  -> WAL frames written to shared PVC
  -> Replicate sidecar reads WAL
  -> Frames uploaded to S3 (1s interval)
  -> Optional: age encryption before upload

Pod restart:
  -> Init container runs replicate restore
  -> Fetches latest snapshot + WAL segments
  -> Reconstructs SQLite database
  -> Primary container starts with warm data
```

2.3 K8s Sidecar Pattern

Each service pod contains three components:

1. **Init container** (`replicate-restore`): Runs `replicate restore -if-db-not-exists -if-replica-exists` to restore from S3 on cold start. No-op if database already exists on the PVC.

2. **Primary container**: The service binary (e.g., `zoo-bridge`, future Base-powered services). Reads and writes `/app/data/data.db`.

3. **Sidecar container** (`replicate`): Runs `replicate replicate` to continuously stream WAL changes to S3. Shares the `/app/data` volume with the primary.

2.4 S3 Path Layout

```
s3://zoo-data/
  {pod-name}/
    generations/
      {generation-id}/
        snapshots/
          {index}.snapshot.lz4
        wal/
          {index}.wal.lz4
```

Each pod (e.g., `zoo-bridge-0`, `zoo-bridge-1`) replicates to a distinct prefix. The `POD_NAME` environment variable is injected via the Kubernetes downward API (`metadata.name` fieldRef).

3 Specification

3.1 Replicate Configuration

Listing 1: replicate-config ConfigMap

```
dbs:
  - path: /app/data/data.db
    replicas:
      - type: s3
        bucket: zoo-data
        path: ${POD_NAME}
        endpoint: ${
            LITESTREAM_S3_ENDPOINT}
        region: sfo3
        force-path-style: true
        sync-interval: 1s
        snapshot-interval: 1h
```

- `sync-interval: 1s` — WAL frames shipped every second, achieving ≤ 1 s RPO.
- `snapshot-interval: 1h` — Full snapshot every hour bounds restore time (restore replays at most 1 hour of WAL).
- `force-path-style: true` — Required for DigitalOcean Spaces and MinIO.

3.2 Encryption

WAL segments and snapshots are encrypted at rest using `hanzoai/age` with per-environment X25519 key pairs:

Listing 2: Encryption configuration

```
dbs:
  - path: /app/data/data.db
    replicas:
      - type: s3
        bucket: zoo-data
        path: ${POD_NAME}
        encryption:
          type: age
          key: ${AGE_SECRET_KEY}
```

Key management:

- Encryption keys stored in Zoo KMS (K-Chain) or Kubernetes secrets (bootstrapping).
- One key pair per environment (devnet, testnet, mainnet).
- Key rotation: generate new pair, re-snapshot with new key, update secret. Old WAL segments remain readable with old key until next snapshot.

3.3 Service Applicability

Table 1: Zoo services and replication targets

Service	Kind	Data	Replicate
zoo-bridge	Deployment	SQLite (future)	Yes
zoo-gateway	CRD (ZooGateway)	Stateless	No
zoo-explorer	Deployment	PostgreSQL	No
zoo-chain	StatefulSet	Node data (PVC)[1]	No
zoo-operator	Deployment	Stateless	No
sql	StatefulSet	PostgreSQL	No
kv	StatefulSet	Valkey	No
ingress	Deployment	Stateless	No

As services migrate from PostgreSQL to Base/SQLite, they gain replicate sidecars with zero code changes — only a K8s patch.

3.4 Recovery Guarantees

RPO $\leq 1\,\mathbf{s}$ WAL frames synced every second. In the worst case, the last second of writes is lost.

RTO $\approx$ **seconds** Init container restores latest snapshot (≤ 1 hour old) plus WAL replay. Typical restore: 2–5 seconds for databases under 1 GB.

Durability S3 provides 11 nines of durability. Snapshots are LZ4-compressed and optionally age-encrypted.

Consistency SQLite WAL mode guarantees atomic commits. Replicate ships complete WAL frames, preserving transaction boundaries.

4 Implementation

4.1 K8s Patches

Patches are applied as standalone YAML files in `k8s/patches/` rather than inline kustomize overlays. This allows selective application per-service and per-environment.

Required patches:

1. `replicate-config.yaml` — Shared ConfigMap
2. `replicate-bridge.yaml` — Bridge deployment patch (when migrated to SQLite)

4.2 S3 Service

Deploy `hanzoai/s3` as a StatefulSet with block storage:

Listing 3: S3 service (MinIO)

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: s3
spec:
  replicas: 1
  template:
    spec:
      containers:
        - name: s3
          image: ghcr.io/hanzoai/s3:
              latest
          ports:
            - containerPort: 9000
          env:
            - name: MINIO_ROOT_USER
              valueFrom:
```

[1] Chain node data replication uses PVC snapshots and chain sync, not WAL streaming. Replicate is for SQLite only.

```
                secretKeyRef:
                  name: s3-credentials
                  key: ACCESS_KEY
            - name: MINIO_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: s3-credentials
                  key: SECRET_KEY
```

4.3 Apply Script

A shell script (`scripts/apply-replicate.sh`) orchestrates the full deployment:

1. Verifies `s3-credentials` secret exists
2. Applies the shared ConfigMap
3. Patches each target service
4. Restarts patched workloads
5. Waits for rollout completion

5 Comparison with Alternatives

Table 2: Replication approaches

Approach	RPO	RTO	Complexity	Cost
PVC snapshots	hours	minutes	low	low
PostgreSQL streaming	0	seconds	high	high
WAL to S3	**1 s**	**seconds**	**low**	**low**
CockroachDB	0	0	very high	very high

WAL-to-S3 replication provides the best tradeoff for Zoo's requirements: near-zero RPO, fast recovery, minimal operational complexity, and compatibility with the Base/SQLite-per-org architecture.

6 Cross-Ecosystem Compatibility

The `ghcr.io/hanzoai/replicate:latest` image is shared across all ecosystems:

- **Lux**: IAM, KMS, Gateway, Commerce (4 services)
- **Lux**: Future Base-powered services
- **Zoo**: Bridge and future Base-powered services

The only per-ecosystem differences are:

- S3 bucket name (`zoo-data` vs `zoo-data`)
- S3 endpoint (GCS vs DigitalOcean Spaces)
- Region (`us-central1` vs `sfo3`)
- Age encryption key pairs

All configuration is injected via ConfigMap and secrets. Zero code divergence between ecosystems.

7 Security Considerations

1. **S3 credentials**: Stored in Kubernetes secrets, never in ConfigMaps or container images. Accessed via `LITESTREAM_ACCESS_KEY_ID` and `LITESTREAM_SECRET_ACCESS_KEY` environment variables.
2. **At-rest encryption**: WAL segments encrypted with `hanzoai/age` (X25519 + ChaCha20-Poly1305). Even if the S3 bucket is compromised, data is unreadable without the age secret key.
3. **Per-org isolation**: Base already encrypts each org's SQLite shard with a unique DEK derived via HKDF. Replicate streams the encrypted bytes — it never sees plaintext org data.
4. **Network policy**: The replicate sidecar only needs egress to the S3 service (`s3.zoo.svc.cluster.local:9000`). A NetworkPolicy can restrict this.
5. **Key rotation**: Age key rotation requires a re-snapshot. The apply script can trigger this via `replicate snapshots create`.

8 Post-Quantum Security

The Zoo ecosystem has achieved comprehensive post-quantum (PQ) coverage across all cryptographic layers, consistent with LP-102 (Lux) and HIP-0302 (Hanzo). All previously planned mitigations are now deployed.

8.1 PQ Scorecard

Table 3: Complete post-quantum scorecard. All layers PQ-safe except EVM chain signing.

Layer	Algorithm	PQ Status	Status
Disk encryption	AES-256 (sqlcipher)	Safe (128-bit PQ)	Deployed
HKDF derivation	HKDF-SHA-256	Safe (128-bit PQ)	Deployed
Field encryption	AES-256-GCM	Safe (128-bit PQ)	Deployed
S3 backup	age ML-KEM-768+X25519 (`age1pq`)	Safe (FIPS 203)	Deployed
TLS	X25519MLKEM768 (first curve)	Safe (hybrid PQ)	Deployed
JWT signing	ML-DSA-65 (FIPS 204)	Safe (Module-LWE+SIS)	Deployed
JWKS validation	ML-DSA-65	Safe	Deployed
MPC transport	Hybrid KEM (X25519+ML-KEM-768)	Safe	Deployed
MPC key shares	PQ KEM encryption	Safe	Deployed
Master keys	Cloud HSM (FIPS 140-2 L3)	Hardware isolation	Deployed
Chain signing	secp256k1 ECDSA	**Not PQ-safe**	EVM constraint

8.2 Deployed PQ Components

S3 backup: All age encryption uses ML-KEM-768 + X25519 hybrid recipients (`age1pq` prefix). Label enforcement prevents downgrade to classical-only recipients.

TLS: X25519MLKEM768 as first curve in TLS 1.3 supported groups.

JWT signing: ML-DSA-65 (FIPS 204) for IAM token issuance. JWKS validation uses ML-DSA-65 public keys.

MPC: Hybrid KEM for transport; PQ KEM encryption for key shares.

HSM: Cloud HSM (FIPS 140-2 L3) for master keys. Key material never leaves HSM boundary. Three keyrings per ecosystem.

8.3 Harvest-Now-Decrypt-Later: Closed

The combination of ML-KEM-768 hybrid encryption (S3), X25519MLKEM768 TLS (transit), ML-DSA-65 JWT signing (auth), PQ KEM for MPC, and Cloud HSM for master keys closes the harvest-now-decrypt-later attack vector completely. An adversary who captures data today cannot decrypt it with a future quantum computer.

9 References

References

[1] Z. Kelling, "Web-of-Trust: Local-First Decentralized Applications with Chain-Anchored Trust," Zoo Labs Foundation, April 2025.

[2] B. Johnson, "Litestream: Streaming Replication for SQLite," `https://litestream.io`, 2021.

[3] F. Valsorda, "age: A simple, modern and secure file encryption tool," `https://age-encryption.org`, 2019.

[4] Zoo Labs Foundation, "ZIP-014: Zoo KMS Integration via Lux KMS," 2025.

[5] Zoo Labs Foundation, "ZIP-015: Zoo L2 Chain Architecture," 2025.

[6] M. Connolly, "X-Wing: The Hybrid KEM You've Been Looking For," Internet-Draft, IETF CFRG, `https://www.ietf.org/archive/id/draft-connolly-cfrg-xwing-kem-05.html`, 2024.

[7] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)," FIPS 203, August 2024.

Status: Draft
Type: Infrastructure
Category: Core
Requires: ZIP-014, ZIP-015
References: LP-102, HIP-0302